Assignment 3

ML Data Product

Date: 08.11.2024

Group 1

Student Last Name	Student First Name	Student ID	Group Allocation
Faiyaz	F M Farhan	24701183	Student A
Li	Yee Ting	24866954	Student B
Sakib	Md Sadman	25091570	Student C
Sovann	Ratana	25396324	Student D

GitHub	Project Repo: <https://github.com/FarhanFaiyaz/adv_mla_at3.git> Streamlit App Repo: <https://github.com/Ratana-Sovann/adv_mla_at3_app>
URLs	Backend: https://at3-app-backend-latest.onrender.com/ Frontend: https://at3-app-frontend-latest.onrender.com/

36120 - Advanced Machine Learning Application
Master of Data Science and Innovation
University of Technology of Sydney

Table of Contents

1. Executive Summary **2**
2. Business Understanding **3**
a. Business Use Cases 3
3. Data Understanding **4**
4. Data Preparation **5**
5. Modeling **6**
a. Approach 1 6
b. Approach 2 6
c. Approach 3 6
6. Evaluation **8**
a. Evaluation Metrics 8
b. Results and Analysis 8
c. Business Impact and Benefits 8
d. Data Privacy and Ethical Concerns 9
7. Deployment **10**
8. Collaboration **11**
a. Individual Contributions 11
b. Group Dynamic 11
c. Ways of Working Together 11
d. Issues Faced 12
9. Conclusion **13**
10. References **14**

1. Executive Summary

The project aims to build multiple predictive models for consumers in the USA to better estimate their interstate airfare. This will enable the stakeholders to gain insights into the overall spending pattern and price sensitivity of the consumers based on the airfare price.

Several algorithms were employed to build and test predictive models. Each group member trained and tested a distinct algorithm for different batches of the dataset (segmented by origin airport). The performance of the algorithms were relatively similar. Afterwards, the models were combined together in a single backend and deployed on Render as a web application for users to predict the outcome based on several inputs.

The predictive models can be acceptable since the performance was in the range of median sales (approx. 25% deviation), however it boils down to the decision of the stakeholders regarding the acceptable margin of error. There is definitely room for improvement with additional feature engineering and fine tuning.

2. Business Understanding

a. Business Use Cases

Objective

The objective of this project is to aid users in the USA better estimate their local travel airfare across multiple states. This user-centric tool will allow travelers to input key details of their trip, such as origin, destination, departure date, time, and cabin type, and receive a predicted flight fare. The ultimate goal is to enhance users' travel planning experience by providing reliable airfare predictions. Below are the primary business cases for this product:

- **Data-Driven Decision Making:** Provide stakeholders with insights derived from the predictive analytics, enabling enhancement in pricing strategies, route planning and demand management.
- **Customer Insights**: Develop a deeper understanding of pricing patterns and sensitivity, allowing for more targeted marketing efforts and personalised promotions.
- **Route Performance Analysis:** Facilitate comparisons of revenue potential and demand patterns across different routes, allowing room for optimisation in the future.

Challenges and Opportunities

CHALLENGES	OPPORTUNITIES
• **Large Dataset:** The dataset was fairly large with created an initial problem processing it. However it was later divided between the group based on airports which made the process doable. • **External Factors:** Local holidays or economic conditions are not fully reflected or explained thereby trends might not be captured fully.	• **Scaleability:** The model can be scaled up if new airports are taken into consideration. Also this can be scaled up for international travels as well. • **Real-time pricing strategies:** There is an opportunity to model real-time pricing strategies to provide users with up-to-date predictions. • **Feature Engineering:** Deriving useful features (eg., peak vs off-peak travel) can improve the model's predictive power.

b. Key Objectives

The project aims to build multiple predictive models using the CRISP-DM framework and agile methodologies. The model predicts the total airfare for domestic flights in the USA. Each model will focus on estimating the expected fare based on key inputs such as origin airport, destination airport, departure date and time, and cabin types. The models will use RMSE as the primary evaluation metric, with the objective of minimising RMSE scores as well as the difference between the training and validation RMSE values to reduce overfitting.

Upon completion, the models will be deployed in production on a Streamlit web application, where users can input trip details and receive predicted fares as output. Each member will contribute to a unique model based on an equally distributed origin airport (discussed in modelling section) and provide users with predictions for their travel plans.

Stakeholders	Role	Influence	Requirement	Engagement Strategy
Travelers/Users	End-users of the airfare prediction app	Low to Medium	User-friendly interface and accurate fare predictions	User testing, surveys, and feedback sessions
Data Science Team	Model development and evaluation	High	Access to historical airfare data and model performance metrics	Regular team meetings and data review sessions
Product Managers	Oversee app development and deployment	High	Insights on model performance and user engagement	Progress updates, performance reports, and feedback sessions
Marketing Team	Promotes app and manages user acquisition	Medium	Understanding of app features and unique selling points	Collaborative sessions for feature overview and marketing insights
Technical Support Team	Maintains app and resolves user issues	Medium	Clear documentation and model deployment details	Training sessions and continuous support availability
Company Executives	Sponsors and strategic decision-makers	High	Insights into model impact and return on investment	Strategic updates and high-level performance reporting

Table: Stakeholder Analysis

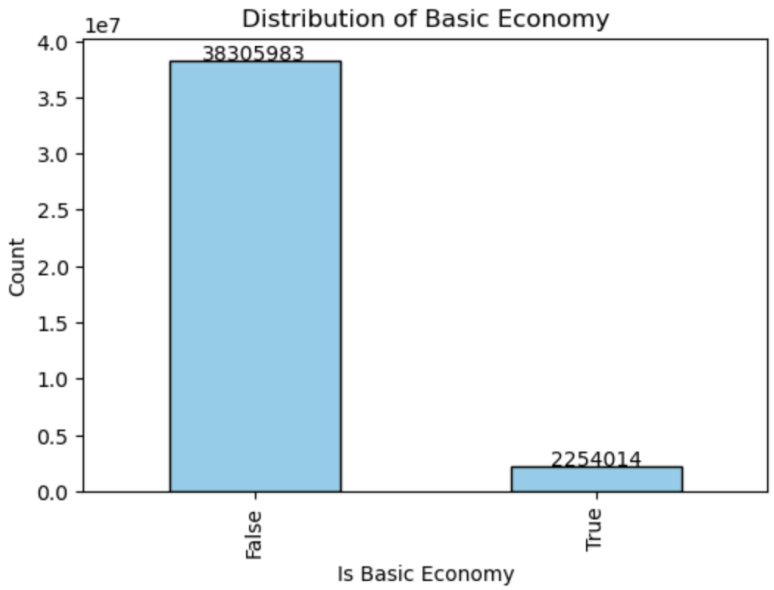
The stakeholder analysis shows the stakeholder requirements and the corresponding strategies using the model to meet these requirements.

3. Data Understanding

The dataset contains information about 16 airports from different states in the USA, with 16 unique destinations. It consists of 40 million rows and 23 features.

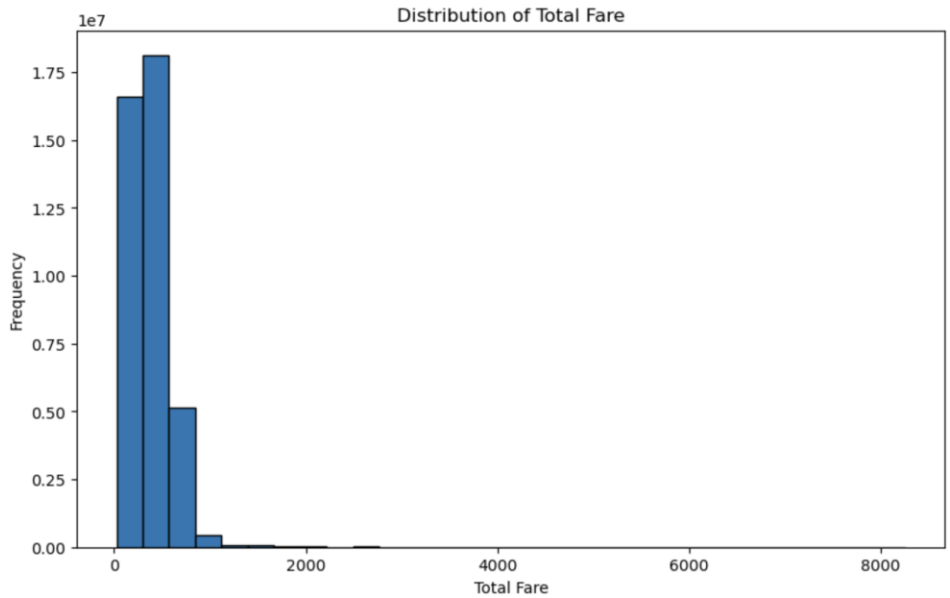
Fetures name	Data Type	Missing values (%)
legId	Object	0
searchDate	Object	0
flightDate	Object	0
startingAirport	Object	0
destinationAirport	Object	0
travelDuration	Object	0
isBasicEconomy	boolean	0
isRefundable	boolean	0
isNonStop	boolean	0
totalFare	float	0
totalTravelDistance	float	7.1
segmentsDepartureTimeEpochSeconds	Object	0
segmentsDepartureTimeRaw	Object	0
segmentsArrivalTimeEpochSeconds	Object	0
segmentsArrivalTimeRaw	Object	0
segmentsArrivalAirportCode	Object	0
segmentsDepartureAirportCode	Object	0
segmentsAirlineName	Object	0
segmentsAirlineCode	Object	0
segmentsEquipmentDescription	Object	1.95
segmentsDurationInSeconds	Object	0
segmentsDistance	Object	0.94
segmentsCabinCode	Object	0

Figure 1.Distribution of Basic Economy



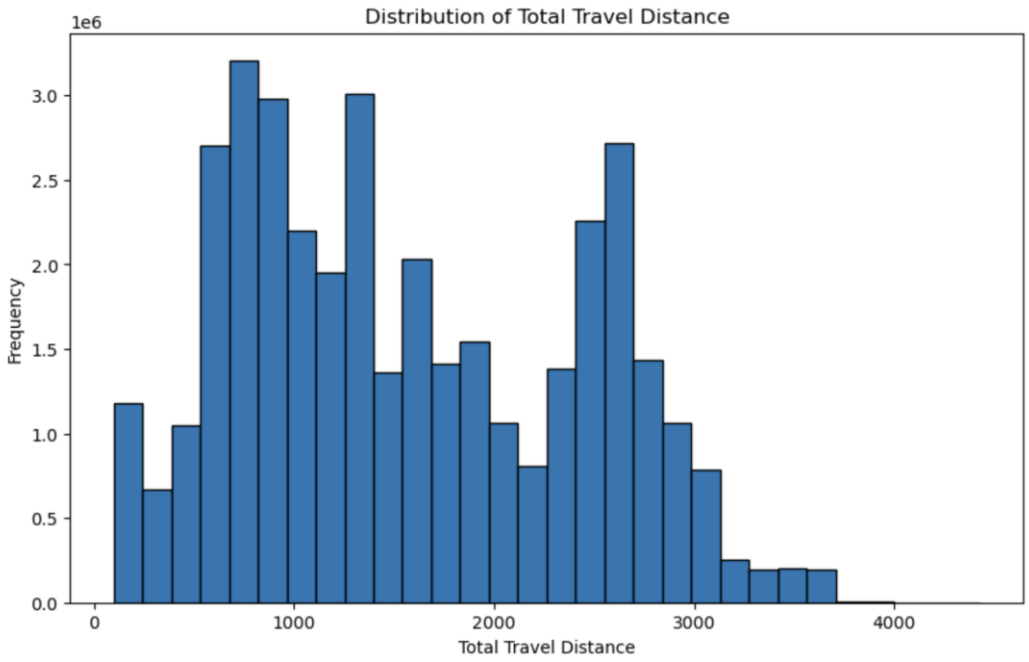
The vast majority of flights in the dataset are not basic economy tickets. Only a small portion of flights are basic economy tickets, which cover about 5.6% of total air traffic in the dataset.

Figure 2.Distribution of Total Fare



The dataset fares are mostly in the lower price range. This distribution shows a slight right skew. There are few flights that are expensive, suggesting these are business class or first-class flights compared to standard-priced tickets. The maximum fare is 8,260, whereas the minimum fare is 239.

Figure 3.Distribution of Total travel Distance



The travel distance distribution shows that maximum travel frequency occurs in the range of 800 to 1,300 miles, and another peak is visible around 2200 to 2400 miles. There are some long-haul flights here, which explains why some flights exceed the distance of 2000 miles.

4. Data Preparation

Handling Duplicates

During the initial exploration, over 60% of duplicates were found in datasets. The duplicates mainly resulted from multiple searches with exact same flight details on the same date. They were removed to ensure the accuracy and reliability of model training.

Handling Missing Values

There were roughly 5% of missing values in totalTravelDistance across all data. The issue was addressed by imputing the median values of distance by aggregating startingAirport and destinationAirport.

Extracting Date and Time Features

Date_diff was computed from the difference between search date and flight date, as it would be an indicator in predicting the target. Day, day_of_week, week_of_year, and month were extracted from the origin departure date. The origin departure time features, including hour and minute, were retrieved after formatting the segmentsDepartureTimeRaw.

Number of stops

n_stops, extracted from segmentsDepartureAirportCode. indicated the number of legs for each flight, for 0 representing direct flight. This can allow users to select their own choices. The original attribute isNonStop was then eliminated .

Extracting Cabin Type

The cabin type of each leg was split from segmentsCabinCode, where ‘no_stop’ represented null and non-existent legs.

4.1 Student A (Farhan)

The data preparation was performed in three stages.

Stage 1:

In the first stage of data processing, the files were loaded into a dataframe using a python script and some introductory transformations were made as shown in the flowchart below.

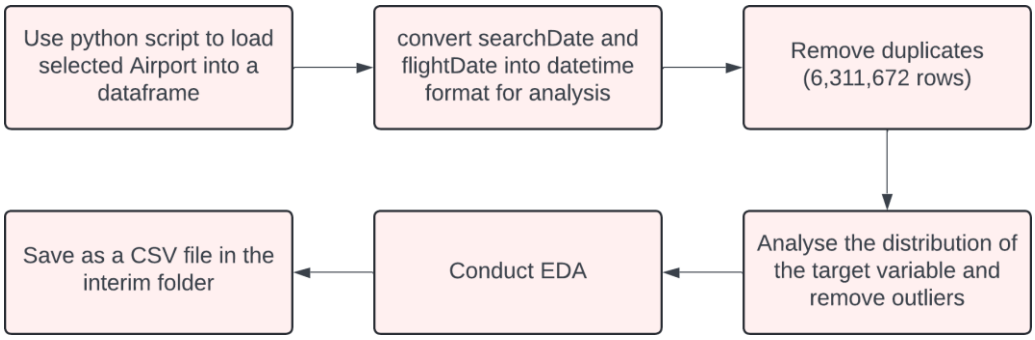


Fig: Data Prep stage 1

The diagram below shows the distribution of the target variable (totalFare). From the box plot, we can see that the median fare is at the lower end (408). There are several datapoints which are beyond the upper whisker that go beyond 2000 dollars with some reaching as high as 8000 dollars. There were about 2,850 rows which were outliers (1.35% of data). Since the objective was to predict typical fares, outliers were removed.

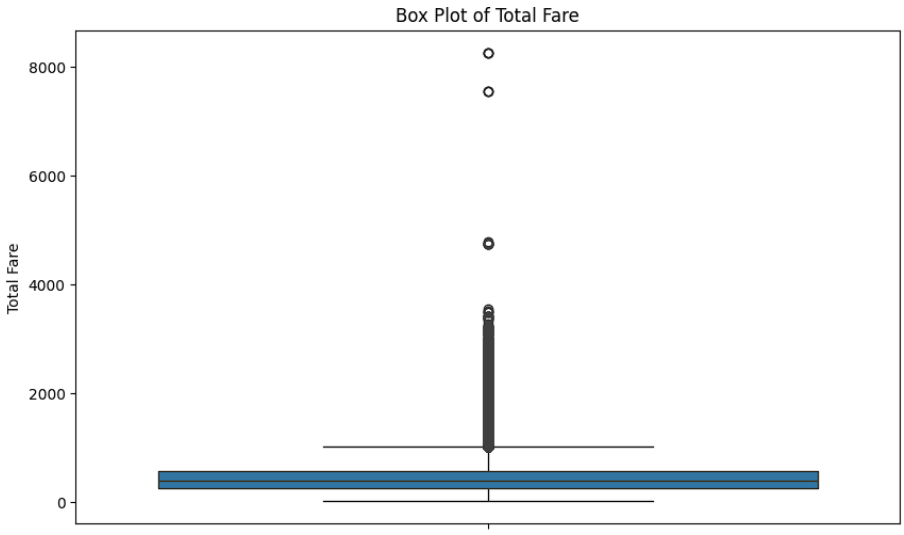


Fig: box plot for TotalFares

Stage 2:

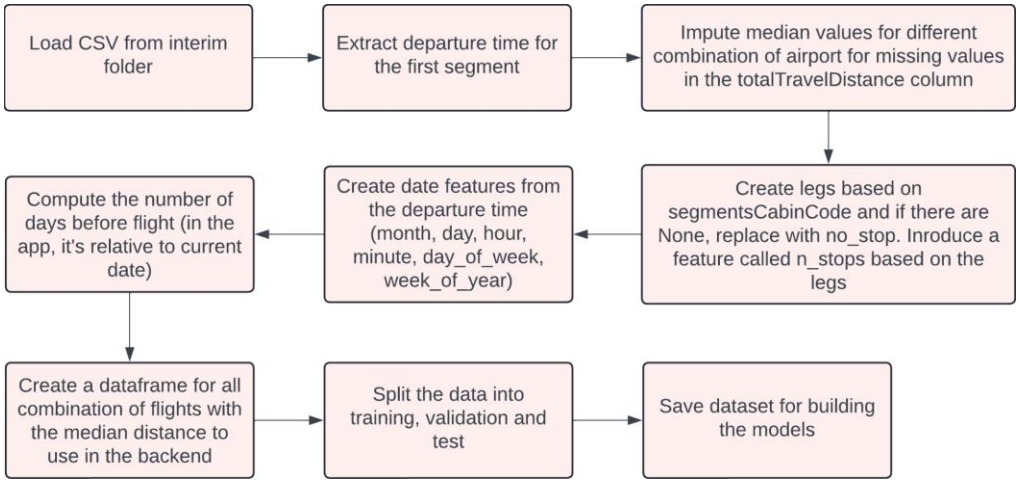


Fig: Data Prep stage 2

In this stage some further transformations and feature engineering were done (mentioned in the common transformation in the first part of this section).

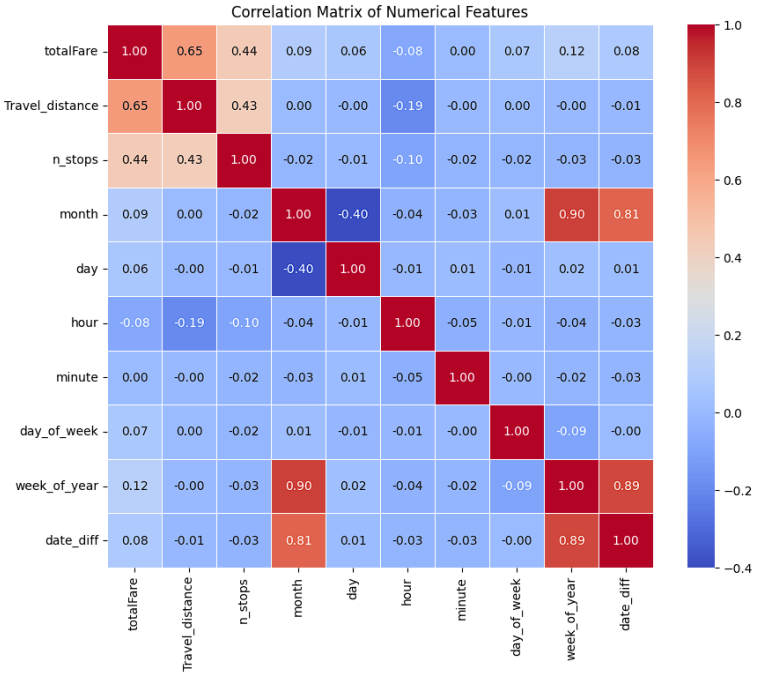


Fig: Correlation matrix

Based on the correlation matrix above, we can see that TotalFare has a relatively high positive correlation with travel_distance and n_stops. Since we cannot directly take the travel distance from the user input, different combinations of flights were listed with their median values and stored in a CSV file which will be later used in the backend integrated with the ML model. Multiple date features were also generated.

The data was split into training, validation and test in the ratio 70:15:15 and was done with respect to the search date to ensure the data is split chronologically since we will be predicting fare for a future date with respect to the current date.

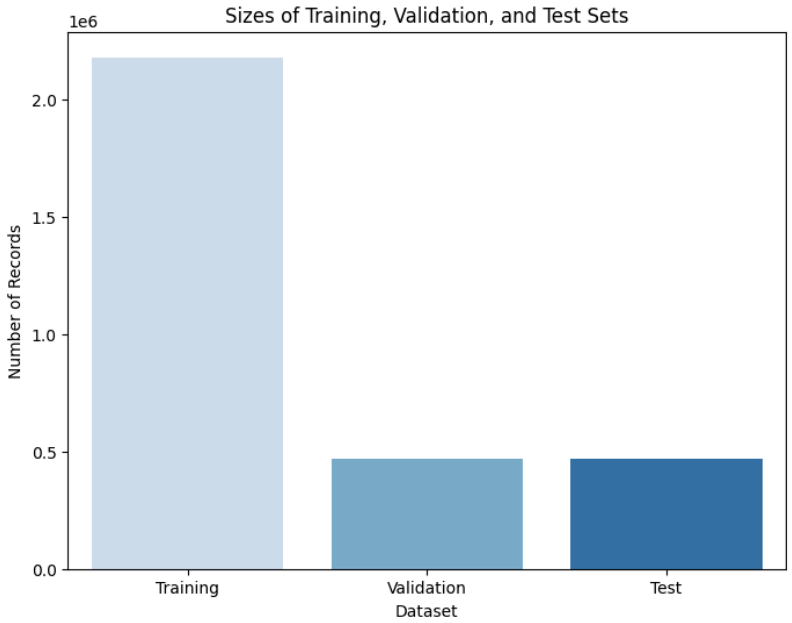


Fig: Ratio of training, validation and test sets

Stage 3:

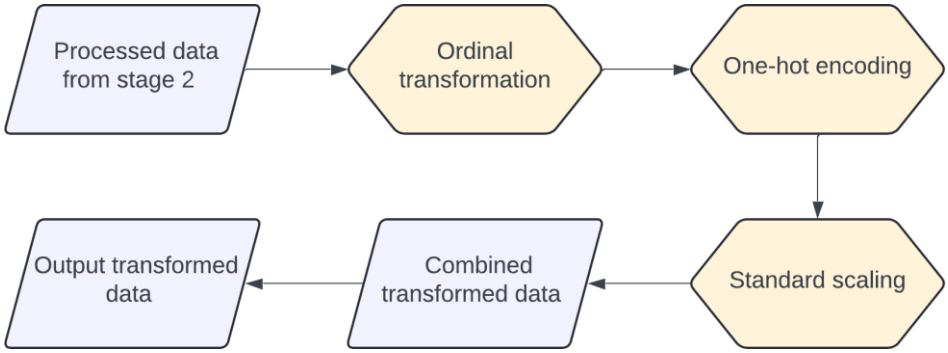


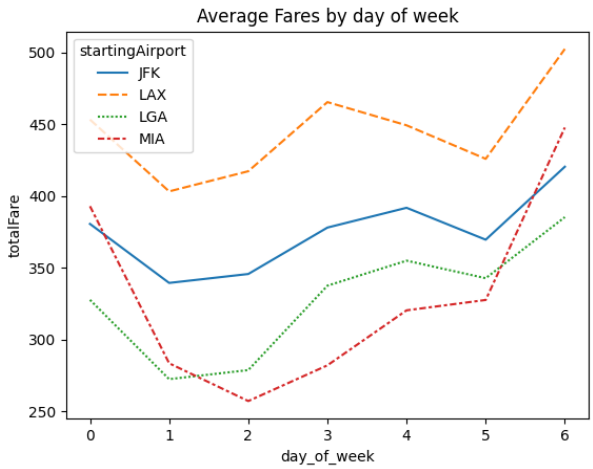
Fig: Data Prep stage 3

In the third stage, the preprocessed data from stage 2 was used to build a pipeline where the cabins were ordinally transformed since the cabin classes have increased prices as they go from coach to first class. Afterwards, the startingAirport and destinationAirport were transformed using one hot encoding and the numerical variables were standardised using standard scaling. The transformed data was used to build the ML models

4.2 Student B (Yee Ting Li)

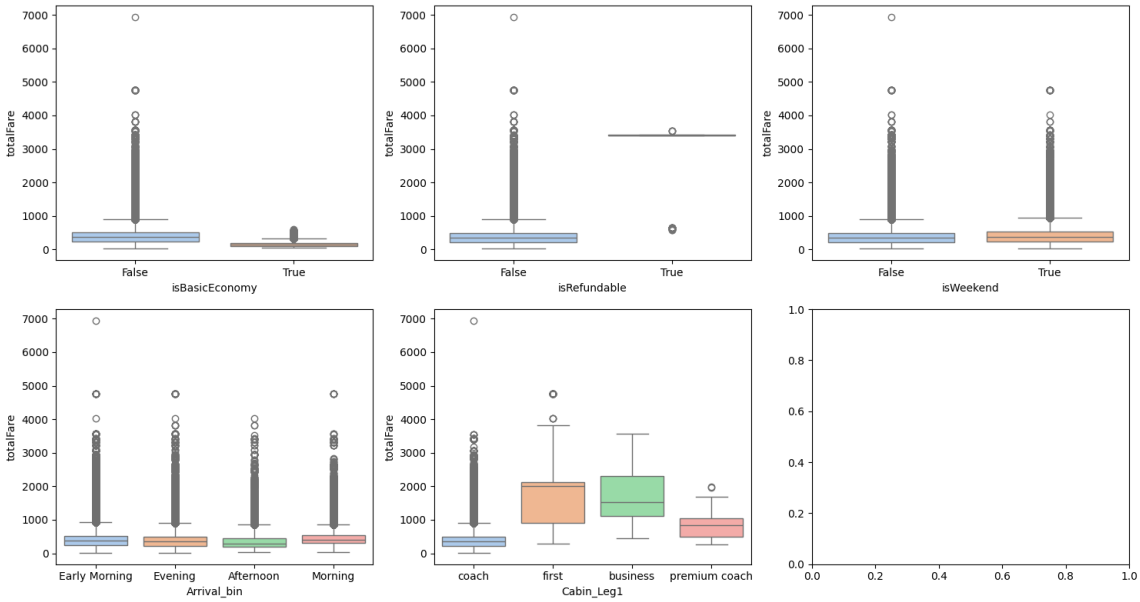
Date Feature

The hypothesis testing would be higher fares for weekend flights. The Boolean feature isWeekend was derived from flightDate. The line chart below depicted a rising trend of fares during the weekend.



Outliers

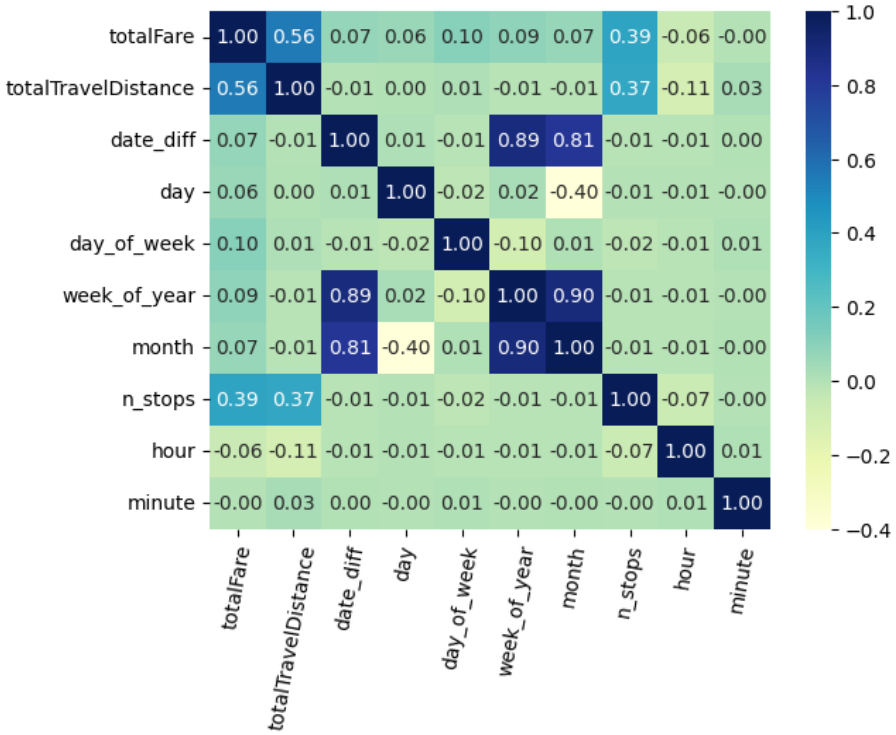
Boxplots were used to examine outliers. As there are many outliers existing in data, removing them would distort the data distribution and lead to biased algorithms and predictions. Therefore, they were maintained during model training.



Feature Selection

Arrival time was extracted initially and categorized into bins. After examining the correlation coefficients, it was dropped for having minimal impact on predicting the target. The isRefundable feature demonstrating an extremely imbalanced proportion was excluded after training the first model.

Below heatmap highlighted positive correlations between totalTravelDistance and n_stops against the target totalFare respectively.



Data Pipelines

The scikit-learn Pipeline and ColumnTransformer were employed to build the data pipelines. Numeric features were transformed into similar scales utilizing StandardScaler. The origin cabin type was transformed into OrdinalEncoder, meanwhile, other categorical features were handled with OneHotEncoder. The high cardinality of destinationAirport was encoded with TargetEncoder.

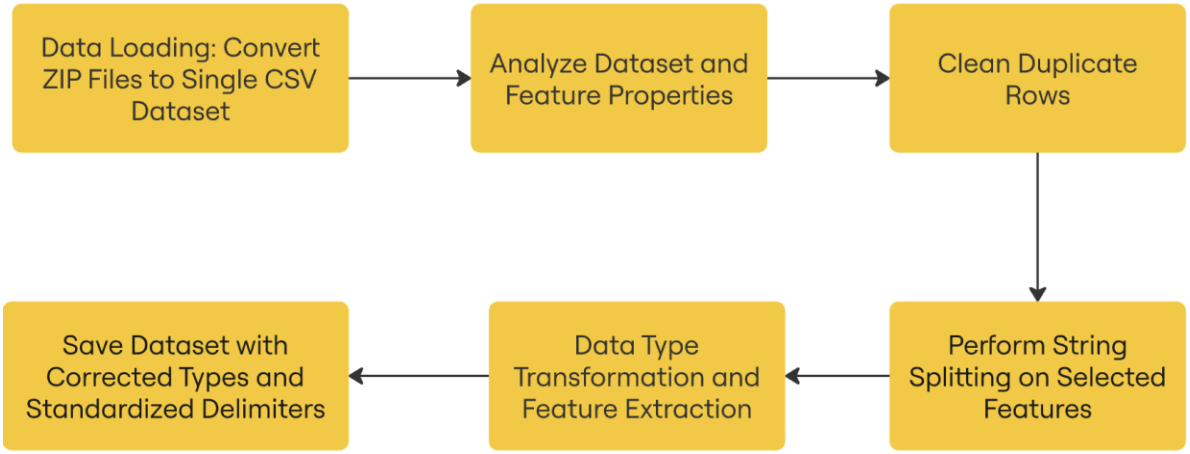
Data Partitioning

Dataset was split into 60-20-20 for training, validation and test set, applying the train_test_split method. This can validate the model performance before making predictions on test data.

4.3 Student C (Md Sadman)

Stage 1 :

To prepare the data for modeling, the processing was divided into two steps. There are 16 airports, and this analysis focuses on 4 airports. All ZIP files were converted into CSV files, then merged into a single consolidated CSV file. After loading the dataset, the process followed the complete flow chart in the first steps. Before removing duplicate legIds, a random sample of 5 legIds was selected to check if their travel durations were consistent across duplicates. The prepared dataset was then saved for the second stage of data preparation.



Stage 2:

After the first steps of data processing, started to go through every feature. Extracted flight information and created new features. There is correlation between fare price and duration, and it is not that high, around 0.31. Removed missing values that are less than 1% and filled travel duration with mean value based on the data distribution. Created date features from the departure time. Made a date difference feature based on the booking date and flight time. Extra features have been created based on the cabin classes. Stored median travel distance and duration between airports pairs and saved as CSV file to work with future backend.

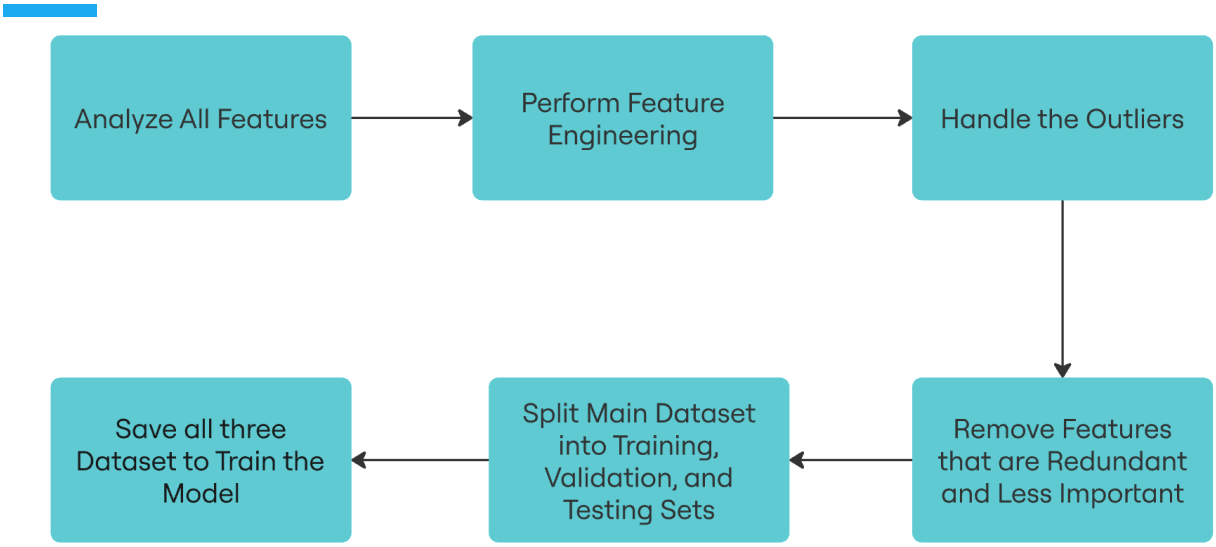
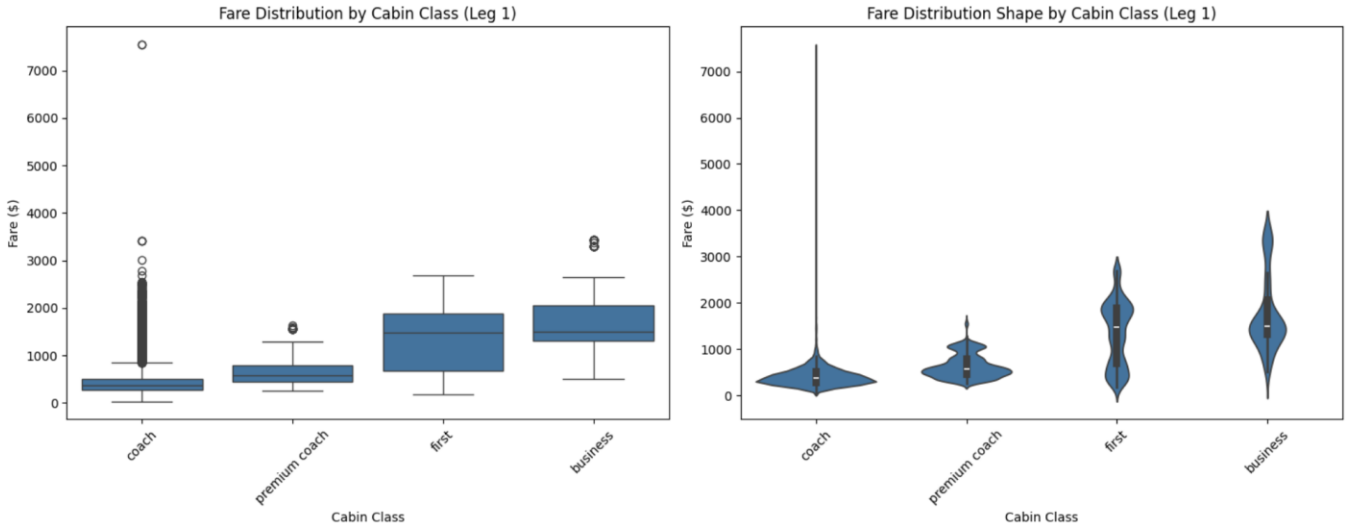


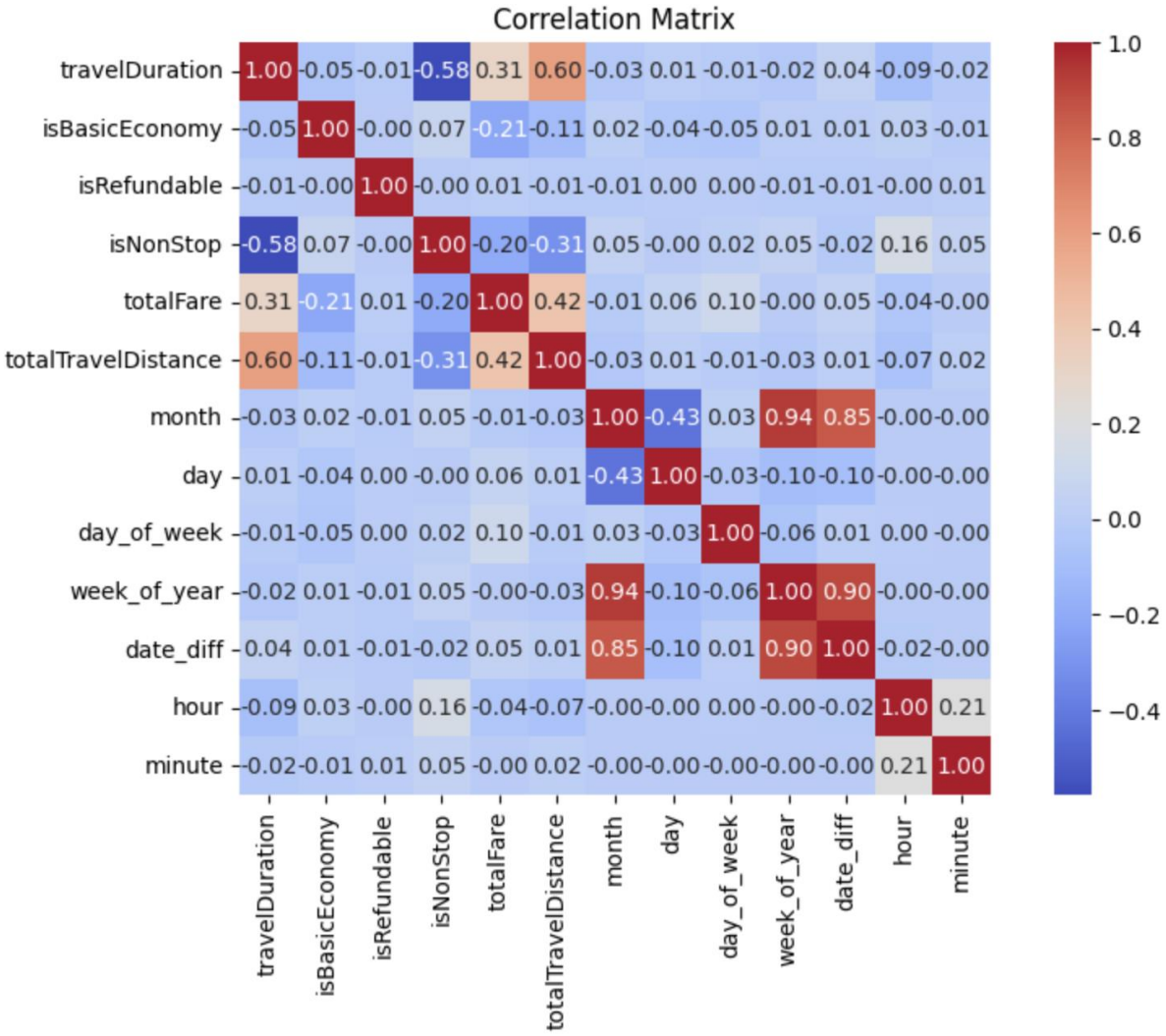
Figure 4. Distribution and Outliers of Fare Prices Across Cabin Classes



Multiple fares exceed the fares of business class and first class. Certain coach fare prices are similar to first class and business class prices. These might be due to last-minute bookings. To capture the last-minute booking pattern, we are considering those fares and removing fare prices that are higher than business-class fares as they are mainly outliers.

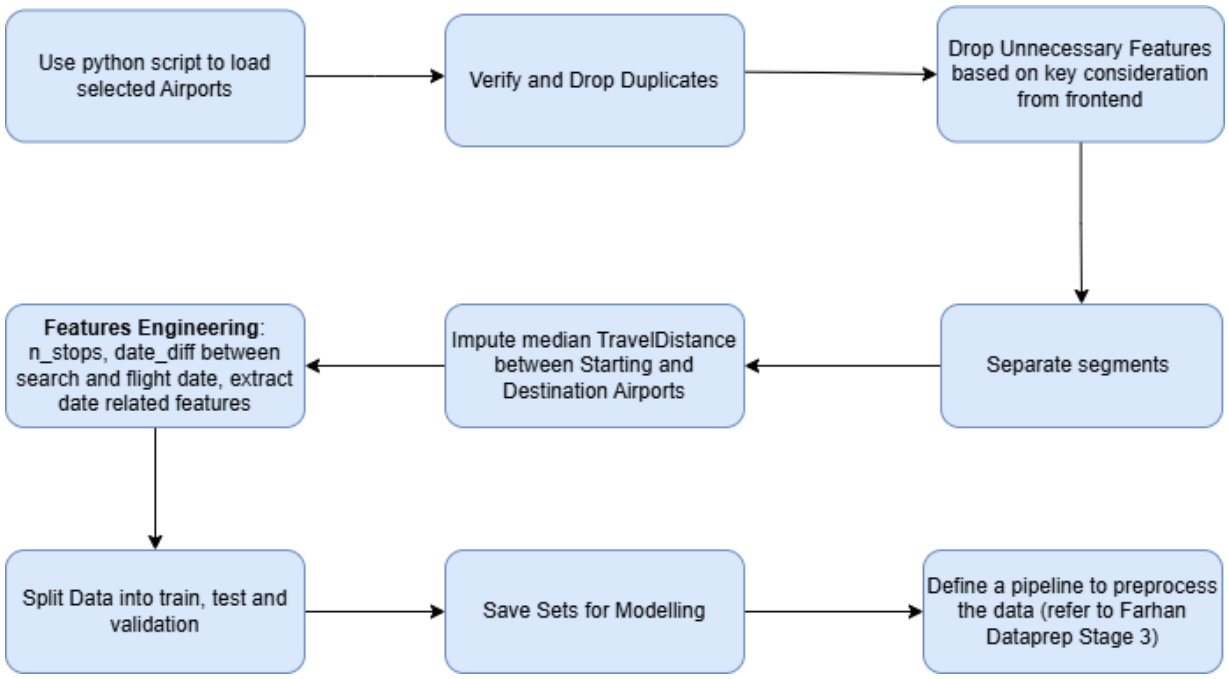
After removing redundant and less correlated duplicated features, split the main dataset into train, validation and test sets.

Feature selection:



4.4 Student D (Ratana Sovann)

The data preparation steps closely mimic my peers above as the features used for modelling are quite similar and are surmised as follows:



The only key differences are some model design choices:

- Is_weekend is not included as day_of_week features already capture the trends.
- Flight duration was not used as I believe it is heavily inflated by layover times which user cannot directly know/specify.
- Outliers were not removed as I believe it contain useful information for modelling.

The final features used for modelling are:

- Month, day, hour, minutes, day_of_week, week_of_year, date_diff, startingAirports, destinationAirports, n_stops, medianTravelDistance, choices of cabins for each legs of trips.

5. Modeling

5.1 Student A (Farhan)

a. Approach 1

In the first approach, a simple linear regression model was built to initially test out the performance and deploy the model quickly with other team members to see if the combined backend works or not. In this stage, the totalTravelDistance was not taken into account in the ML model.

ML ALGORITHM	RATIONALE	HYPERPARAMETER(S) TUNED
Linear Regression	Basic and widely used model for predicting continuous outcome such as TotalFare. The objective of this approach is to integrate multiple models and deploy it to check any underlying issues.	Linear regression with default parameters were used. No hyperparameters were tuned.

The model was integrated seamlessly with Ratana and Sharon's models thereby indicating that new algorithms could be tried out and hyperparameters could be tuned. Additionally it performed better than the baseline model indicating that the modelling was in the right track.

b. Approach 2

In this approach, the same linear regression model was built but the new totalTravelDistance feature was introduced since it had high correlation with the totalFare. As mentioned earlier, a csv file with all the combination of airports with their median distances were computed and used in the backend.

ML ALGORITHM	RATIONALE	HYPERPARAMETER(S) TUNED
Linear Regression	To compare the model with approach 1 after introducing the totalTravelDistance feature	Linear regression with default parameters were used. No hyperparameters were tuned.

Introducing the new feature and keeping all other variables constant led to an improvement in the performance.

c. Approach 3

In the final approach 6 different ML algorithms were built and multiple iterations were performed. Individual pipelines were built for each model and hyperparameters were tuned inside the pipelines. The ML algorithms, rationale and hyperparameters tuned are shown below:

ML ALGORITHM	RATIONALE	HYPERPARAMETER(S) TUNED
Ridge Regression	Ridge Regression applies L2 regularisation to linear regression which adds a penalty proportional to the square of the coefficients.	Alpha was tuned. Alpha = 1, 0.5, 2 were taken. Tuning it did not impact the output much. The performance is similar to the linear regression model built in approach 2.
Lasso Regression	Lasso Regression uses L1 regularisation which can shrink unimportant coefficients to zero.	Alpha = 1, 0.5 were taken. Tuning the hyperparameters did not impact the output much either.
Elastic Net Regression	Elastic Net combines both L1 and L2 regularisations including both penalties.	Alpha = 1, 0.5, 0.1, 0.01, 0 were tuned. L1_ratio= 0.2, 0.8 were also tuned. The output was similar to the previous models.
Stochastic Gradient Descent	SGD is capable of handling large datasets as it updates model parameters incrementally using small random batches of data.	Alpha = 1e-5, 1e-4, 1e-3, 1e-2, 1e-1 were taken and looped for each values. They were relatively same in value but 0.1 increased the RMSE slightly. penalties = l1, l2 and elasticnet were taken which didn't impact the output very much. learning_rate = 'constant', 'optimal', 'invscaling', 'adaptive' were taken. The outputs were similar except optimal where it was unusually high.
Decision Trees	TotalFare is influended by complex, non linear relationships. Decision-trees can model non-linear relationships and interactions between varibales.	max_depth = 10,5, 20, 15 were taken. min_samples_split=5,3 were taken. min_samples_leaf= 5, 10 were taken. model_max_features=auto, sqrt, log2, none were taken. Afterwards, randomserachCV was used to find the best combination of hypereparmaters. 'model__min_samples_split': np.int64(6), 'model__min_samples_leaf': np.int64(2), 'model__max_features': None, 'model__max_depth': np.int64(16)
LightGBM	Optimised for faster training and is more efficient than other boosting algorithms. Since a group member already opted for XGBoost, I decided to try LightGBM. It is also a tree based model that can capture non linear relationships.	max_depth = 3, 10, 20, 30 were tuned. num_leaves = 10, 20, 40, 60, 100, 200, 500, 1000 were tuned. n_estimators = 50, 200, 500, 1000 were tuned. learning_rate = 0.5, 0.01, 0.2 were tuned. Hyperopt was used to find the best combination of hyperparamters to give lower RMSE but the combination was not the best. Afterwards random search algorithm was used which gave a relatively good set of hyperparamters with low RMSE and slight overfitting. 'model__subsample': np.float64(0.8), 'model__num_leaves': np.int64(500), 'model__n_estimators': np.int64(300), 'model__min_child_samples': np.int64(10), 'model__max_depth': np.int64(16), 'model__learning_rate': np.float64(0.2822222222222222), 'model__colsample_bytree': np.float64(0.9)

5.2 Student B (Yee Ting Li)

This section outlines the thorough experimentation process, rationale, hyperparameters tuned for each approach.

a. Approach 1

Linear Regression

- Initially, the simple and interpretable linear regression algorithm was trained.
- The result established as the benchmark for comparing with more complex algorithms

Stochastic Gradient Descent

- Subsequently, the Stochastic Gradient Descent approach was taken.
- Implementing the regularization and loss function can help minimize the error and improve the accuracy of the model's predictability.

b. Approach 2

Adaptive Boosting Regression

- The ensemble learning algorithm AdaBoost was selected, aiming to enhance the accuracy of the model
- By combining multiple weak learners, it can reduce the error and alleviate risk of overfitting.

c. Approach 3

Extreme Gradient Boosting Regression

- The tree-based ensemble learning algorithm XGBoost was chosen, taking the training time and performance into consideration.
- The iterative learning process and gradient descent employed for each tree can minimize the loss function to optimal level.
- It demonstrated the most efficient, consistent and robust performance compared to the aforementioned algorithms trained.

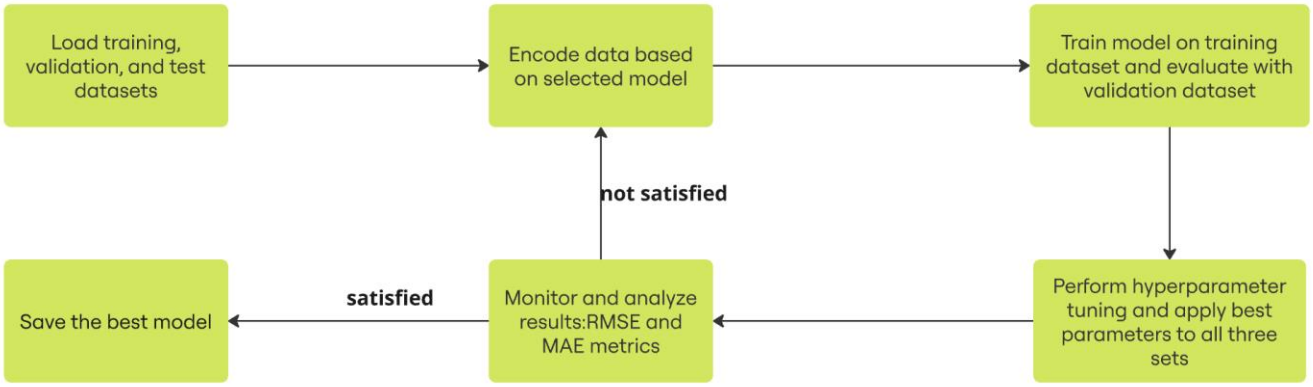
The table below entails considerations of each approach.

ML Algorithm	Rationale	Hyperparameters Tuned
Dummy Regressor	· Baseline model	● Strategy="mean" ● Baseline model ● Fit the model with pipeline
	Approach 1	
Linear Regression	● Efficient and straightforward ● As Baseline performance	● Default parameters ● Given simplicity and interpretability, the algorithm was chosen as the first model for getting initial insights ● Defining as the benchmark performance to compare against more complex algorithms
SGD Regression	● Applying regularization and loss function in order to minimize the loss to the optimal level ● More efficient in handling large dataset compared to Lasso and Ridge regression	**Values Tuned:** ● Loss – 'squared_error', 'huber' ● penalty – 'l1', 'l2', 'elasticnet' ● l1_ratio=0.15, 0.5, 0.8 ● alpha – 0.0001, 0.01 ● max_iter=1000 ● tol=1e-3 ● learning_rate='optimal' ● early_stopping=True ● random_state=42 **Considerations:** ● When tuning the loss function, huber' was used to mitigate the bias for focusing on outliers ● Tuning the penalty with 'elasticnet' can shrink the magnitude of coefficients of less significant features and strike a balance between L1 and L2 regularization ● Higher value of alpha resulted in higher RMSE, and therefore the default 0.0001 was adopted. ● Limiting the maximum number of iterations and early_stopping can help terminate the training if performance is not improving.
	Approach 2	
AdaBoost Regression	● Ensemble learning from weak learners can help improve accuracy in predictions ● By combining multiple weak learners, it can minimize the error sequentially	**Values Tuned:** ● learning_rate – 0.01, 0.1, 0.3 ● n_estimators - 50, 100 ● loss='square' ● random_state=42 **Considerations:** ● A smaller learning rate of 0.01 reduced impact of each weak learner, resulting in lower RMSE. ● The square error was used to update the weight after each iteration. ● However, the model was sensitive to outliers. Since outliers were retained in data, the trained algorithm might be highly prone to outliers and result in biased predictions. Added, it led to overfitting issues. ● In addition, the algorithm enhanced the computational time and cost with the large dataset.

ML Algorithm	Rationale	Hyperparameters Tuned
		Approach 3
XGBoost Regression	● Efficiency in training ● Ensemble learning decision trees ● Capability in handling missing values and scalable data ● Robustness and consistency in model performance demonstrated	**Hyperopt** technique was employed to allow more efficient search and optimize the hyperparameters tuning. **Search Space:** ● max_depth – choice from 3 to 15 by step of 1 ● min_child_weight – choice from 3 to 15 by step of 1 ● learning_rate – quniform from 0.01 to 0.4 by step of 0.05 ● subsample - quniform from 0.6 to 1 by step of 0.05 ● colsample_bytree - quniform from 0.3 to 1 by step of 0.05 ● random_state=42 ● max_evals – 5, 10, 30 **Optimal Values:** ● max_depth = 9 ● min_child_weight = 9 ● learning_rate = 0.030 ● subsample = 0.65 ● colsample_bytree = 0.55 ● random_state=42 **Considerations:** ● The maximum number of optimization run was limited to 10 for balancing runtime and consistent performance ● Limiting the max_depth and min_child_weight can help prevent overfitting ● The algorithm is less sensitive to outliers compared to SGD and AdaBoost ● Enabling random sampling with subsample and colsample_bytree can further mitigate the risk of overfitting.

5.3 Student C (Md Sadman)

This section describes the comprehensive model development pipeline, which implements a systematic approach for model training, hyperparameter tuning, and model selection.



a. Approach 1

Linear Models

Started with Linear Regression as baseline

- Chosen for simplicity and interpretability.
- Helps establish basic performance benchmarks
- Easy to understand feature relationships

Explored regularized variants:

- Ridge: To handle multicollinearity
- Lasso: For feature selection capabilities
- .Elastic Net: Combines benefits of both Ridge and Lasso

ML Algorithm	Rationale	Hyperparameters Tuned
Linear regression	● This model was selected for its simplicity and ease of implementation, helping to establish a basic understanding of how the model works and providing clear insights into feature relationship.	Did not perform any hyperparameter tuning.

Ridge Regression	● Chosen to handle multicollinearity in the dataset ● ● Helps prevent overfitting by adding L2 regularization	Value used tuned this model Alpha value used 0.01,0.1,1.0,10,100 Solver used 'auto', 'svd', 'cholesky', 'lsqr Ridge Best Parameter : alpha': 0.1 and solver ‘ auto’ The performance was similar to the baseline model.
Lasso regression	● Performs L1 regularization which can eliminate less important features ● Beneficial when we want to identify the most important features	Alpha value used 0.01, 0.1, 1.0, 10.0, 100.0 Selection used 'cyclic', and ‘random’ Lasso Best Parameters alpha 0.01 and regressor cyclic The model output is similar to baseline model
Elastic net Regression	● Balances between feature selection and coefficient reduction ● Particularly useful when dealing with multiple correlated features	alpha: 0.01, 0.1, 1.0, 10.0, 100.0 l1_ratio: 0.1, 0.3, 0.5, 0.7, 0.9 ElasticNet Best Parameters: alpha 0.01 and l1_ratio 0.9 The performance did not improve.

b. Approach 2

To capture the complex patterns that simple linear regression was unable to capture, decision tree and random forest models were chosen to train the model. These two models have the capability to capture complex patterns.

ML Algorithm	Rationale	Hyperparameter Tuned
Decision Tree	• Makes splits based on most informative features at each node. • Can capture both linear and non-linear patterns. • Automatically handles missing values. • Easily accommodates both numerical and categorical data.	Hyperparameter tuning performed using GridSearchCV • max_depth: 5, 10, 15, 20, None • min_samples_split: 2, 5, 10 • min_samples_leaf: 1, 2, 4 • max_features: 'sqrt', 'log2', None Best Parameters: • max_depth: None • max_features: None • min_samples_leaf: 4 • min_samples_split: 10 Model started performing well and validation and training performing both showed less error than previous model
Random Forest	• Ensemble learning from multiple trees can help improve accuracy in predictions • By combining multiple trees, it can reduce overfitting through averaging. • More stable than single decision trees,	Random Forest Parameters: • n_estimators: 100, 200 • max_depth: 10, 20, None • min_samples_split: 2, 5 • min_samples_leaf: 1, 2 • max_features: 'sqrt', 'log2' Best parameters : • max_depth: None • max_features: 'sqrt' • min_samples_leaf: 1 • min_samples_split: 2 • n_estimators: 200 The model began to capture complex patterns and showed signs of overfitting. However, its validation performance improved compared to the decision tree.

c. Approach 3

Data cleaning steps were performed to handle outliers: fare prices exceeding 4000 were removed, and outlier values in coach prices were clipped since coach was the most common mode of travel in the dataset. Additionally, outlier values in total travel distances were clipped. Following these preprocessing steps, and tackling the overfitting, the CatBoost model was implemented.

ML Algorithm	Rationale	Hyperparameter Tuned
Catboost	CatBoost handles categorical features automatically without preprocessing. It uses ordered boosting to reduce prediction shift and has built-in techniques to fight overfitting. Since our model is overfitting, CatBoost is the perfect choice to address this issue.	For hyperparameter tuning, Optuna was used to get better results and leverage the power of the GPU. ● Used early stopping = 50 ● iterations: 500 to 2000 ● learning_rate: 0.01 to 0.3 (log scale) ● depth: 4 to 10 ● l2_leaf_reg: 1e-8 to 10.0 (log scale) ● min_child_samples: 5 to 100 ● subsample: 0.5 to 1.0 ● colsample_bylevel: 0.5 to 1.0 ● random_strength: 1e-8 to 10.0 (log scale Best parameter ● iterations = 1734 ● learning_rate = 0.213 ● depth = 10 ● l2_leaf_reg = 1.46e-7 ● min_child_samples = 90 ● subsample = 0.863 ● colsample_bylevel = 0.655 ● random_strength = 0.0997 Reduced the overfitting and achieved stable results in both validation and test datasets

5.4 Student D (Ratana Sovann)

a. Approach 1

Model	Rationale	Hyperparameters Tuned
Linear Regression	Same rationale as my peers, to train the fastest model with all the features required and integrate it with other model for a quick deployment. This is done to identify issues early before diving into more complex model.	No hyperparameters required.

b. Approach 2

ML Algorithm	Rationale	Hyperparameter Tuned
KNN-regressor	The k-NN algorithm predicts the target value of a new data point based on the average (or weighted average) of the target values of its k nearest neighbors in the feature space. The idea is flight fare for the same group (similar destination, distance, route etc) should be within similar range.	• The trained model took a long time to predict. This is likely because the model needs to compare new data points with each sample in training data. As such for our purpose of developing a real time prediction, this model does not fit. Hence it is terminated. Future experiments will look into ensemble techniques.

The KNN model was not successful in predicting flight fare price due to large number of datapoints and dimensions of features.

c. Approach 3

The third approach focuses on algorithms that use ensemble techniques such as Gradient Boosting Machines, Extratree regressors capable of achieving good results for regression tasks.

ML Algorithm	Rationale	Hyperparameter Tuned
XGBoost	XGBoost is ideal for predicting flight fares due to its ability to handle complex, non-linear relationships between various features while maintaining high performance through efficient training and built-in regularization. It balances both performance and speed.	The following hyperparameters was tested manually: • n_estimators = 50, 100, 150 • max_depth = 3,6,9,12 • min_child_weight = 3, 6, 9, 12 • subsample = 0.6, 0.7,0.8,0.9 • colsample_bytree = 0.5, 0.6, 0.8, 1 **Optimal parameters:** • n_estimators = 100 • max_depth = 9 • min_child_weight = 9 • subsample = 0.7 • colsample_bytree = 0.5 **Key learning:** • RMSE range between 80 - 85 before training start to overfit
LightGBM	More efficient in handling large datasets. It uses advanced optimization techniques to achieve a very fast training speed. The superior fast training time enables it to be used in conjunction with GridSearchCV to exhaustively find the best hyperparameters.	The following hyperparameters was tested manually before using GridSearch:: • n_estimators = 50, 100, 150 • max_depth = 5, 10, 20, 30 • learning_rate = 0.1, 0.05, 0.5 • num_leaves= 10, 20 , 30, 50 • colsample_bytree = 0.5, 0.6, 0.8, 1 **Optimal parameters:** • n_estimators = 100 • max_depth = 9 • min_child_weight = 9 • subsample = 0.7 • colsample_bytree = 0.5 **Key learning:** • RMSE range between 80 - 85 before training start to overfit

Extratree	Extratree builds trees in parallel, and each tree is constructed using fully random splits. It requires less hyperparameter tuning and might outperform XGBoost as the dataset available for each airports is only 3 months (relatively small)	The following hyperparameters was tested manually • n_estimators = 100, 150, 200 • max_depth = 5, 10, 15, 20, • min_samples_split= 2, 3, 5 • min_samples_leaf= 1, 2, 4 • max_features= 0.5, 0.6, 0.8, 0.9 **Optimal parameters:** • n_estimators =, 200 • max_depth = 20 • min_samples_split= 3 • min_samples_leaf= 2 • max_features= 0.8 **Key learning:** • RMSE range between 80 - 85 before training start to overfit • Took a longer time to train, hence only fitted with 20% training data to tune hyperparameter

6. Evaluation

a. Evaluation Metrics

Root Mean Squared Error (RMSE) is selected as the primary evaluation metric for assessing the performance of the models. RMSE directly measures the average magnitude of error in the model's predictions providing an interpretable metric for assessing the deviation from actual value.

Given the project's goal is to predict TotalFare as close as possible for individual travel bookings, RMSE serves as a crucial metric since the accuracy of fare predictions directly impacts business decisions related to pricing strategies, forecasting and resource planning. A lower RMSE indicates that the predictions are closer to the actual values, enabling the stakeholders to make informed business decisions.

Lastly, RMSE is expressed in the same units as the target variable, making it easy for stakeholders to interpret in dollar terms and understand the model's average error.

The project's aim is to minimise the RMSE scores as much as possible ensuring the model can predict the totalFare as close as possible to the actual values.

b. Results and Analysis

6.b.1. Student A (Farhan)

Approach 1:

The RMSE Scores of Training and validation for the first approach using the linear regression model is shown below.

Experiment	RMSE		Key Findings During Experiment
Linear Regression	Training	153.30	It can be seen that the RMSE scores are in the same range of the TotalFare indicating that the initial model is somewhat working. The main objective of this model in this approach is to quickly deploy the model.
	Validation	159.55	

The baseline RMSE was 190.93, indicating that the model performs better than the dummy regressor.

Approach 2:

The RMSE Scores of Training and validation for the second approach using the linear regression model is shown below where the new feature totalTravelDistance is included in the model keeping other factors constant.

Experiment	RMSE		Key Findings During Experiment
Linear Regression	Training	132.19	In can be seen that introducing the new feature totalTravelDistance improves the performance of the model
	Validation	135.69	

Approach 3:

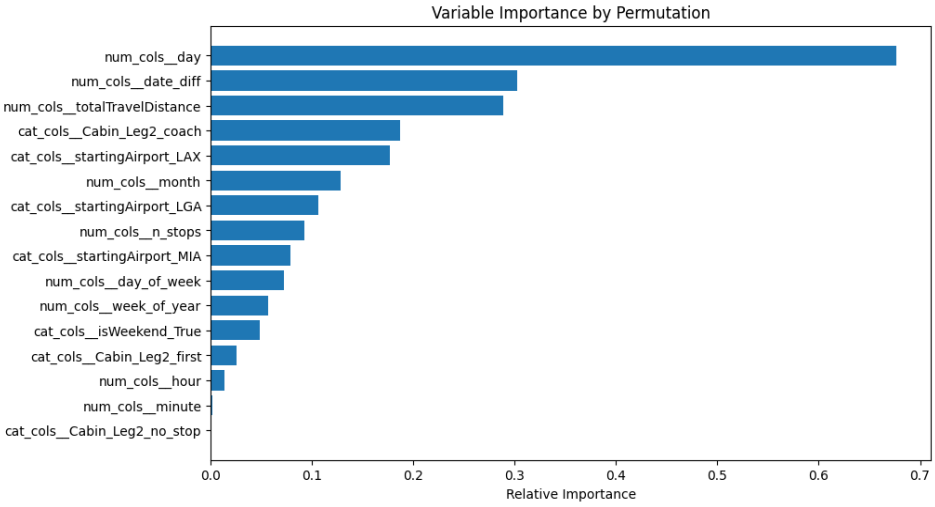
In the final approach, 6 different algorithms were tried out and several iterations were done. The table below shows the best values for training and validation for each algorithm. Out of all the algorithms, LightGBM performed better with slight overfitting.

Experiment	RMSE		Key Findings During Experiment
Ridge Regression	Training	132.18	• The outcome is the same as the Linear regression. Tuning the hyperparametralpha did not impact the RMSE values by much.
	Validation	135.69	
Lasso Regression	Training	132.84	• The performance is slighly poorer compared the Linear Regression and Ridge regression models. Indicates that causing coefficients to be zero is not benefiting the model much.
	Validation	136.71	
Elastic Net Regression	Training	132.20	• The outcome indicates the performance is similar to the linear/ ridge regression with no significant difference RMSE scores.
	Validation	135.79	
Stochastic Gradient Descent	Training	132.19	• The performance is very similar to the previous models.
	Validation	135.74	
Decision Trees	Training	88.16	• Better performance compared to the previous models. • Indicates that tree based model can better capture complex non-linear relationships.
	Validation	97.88	
LightGBM	**Training**	**82.55**	• One iteration of the model gave an RMSE score of 69.56 on the training set but level of overfitting was higher. Thereby model 12 was chosen by sacrificing some performance to lower overfitting. • This model performs slightly better than the Decision trees with slight overfitting.
	Validation	**88.91**	
	Test	**100.44**	

6.b.2. Student B (Yee Ting Li)

ML Algorithm	RMSE		Insights
Dummy Regressor	203.71		● Benchmark error
Linear Regression	Train	150.05	● Relatively consistent performance with moderate overfitting in training data. ● Next approach was to further improve the accuracy of prediction
	Validation	150.76	
	Test	149.93	
SGD	Train	164.35	● Though the model became more consistent across all sets, a higher RMSE resulted. ● Ensemble learning approach would be taken in the next experiment to further minimize the error
	Validation	164.72	
	Test	164.55	
AdaBoost	Train	157.76	● Improvement shown in model performance against SGD ● Yet, computational cost and sensitivity to outliers led to ineffective performance than Linear Regression
	Validation	158.35	
	Test	157.68	
XGBoost (Final model)	Train	89.12	● A more robust model that can predict consistently across seen and unseen data compared to previous models trained. ● Though mild overfitting issues exist, the model has addressed it to an optimal level. As a result, the significant improvement in RMSE can be noted. ● XGBoost was the most effective model, indicating its robust predictability and generalization on future data ● Therefore, the model was chosen to be the final one for deployment.
	Validation	91.53	
	Test	90.75	

Variable importance by permutation



Examining the permutation scores, the above bar chart underscored the day, date_diff and totalTravelDistance to be the top three influential features in predicting the target for the final model chosen.

6.b.3. Student C (Md Sadman)

Approach 1

The first approach result is below.

ML Algorithm	RMSE		Key Findings
linear Regression	training	166.55	This baseline model helps us evaluate the model's performance. It's fast and easy to understand, allowing us to quickly get an idea of how well the model performs. The close RMSE values between training and validation sets suggest stable performance, making it a good starting point for further model development
	validation	172.68	

ML Algorithm	RMSE		Key Findings
Ridge Regression	training	165.75	All three models show similar performance to the baseline model. Despite trying different regularization techniques (Ridge, Lasso, Elastic Net), it did not achieve significant improvement. The consistent RMSE values across models suggest that the problem might be more complex than what simple linear models can handle. Therefore, different models might be able to better capture the underlying patterns in the data
	validation	171.69	
Lasso Regression	training	166.87	
	validation	172.9	
Elastic Net regression	training	168.09	
	validation	174.26	

Approach 2.

After observing stable performance in the first approach using linear regression, tree-based algorithms (decision tree and random forest) were implemented. The results of this second approach are shown below

ML Algorithm	RMSE		Key Findings
Decision Tree	training	68.7	Both tree-based models are performing well on unseen data sets. The validation RMSE decreased from 172 to 117 for the decision tree, and to 97.22 for the random forest. The models are now capturing complex patterns, but have started overfitting. For further analysis, validation datasets were randomly selected to evaluate performance across 5 different datasets. The findings show varying performance: some predictions achieved RMSE between 1 to 2, while others showed RMSE between 80 to 100. There are outliers that need to be removed, as they are likely causing these inconsistent results. While the validation RMSE is lower than the first approach, outlier removal is necessary for improved model performance.
	validation	117.69	
Random Forest	training	34.45	
	validation	97.22	

Approach 3.

After feature engineering removing outliers and clipping boosting algorithm was performed. The results of this third approach are presented below.

ML Algorithm	RMSE		Key Learning
catboost(best model)	training	77.62	CatBoost performed overall better than Random Forest. In training, it showed RMSE score 77.62, and the validation result was 92.61. On the test dataset, it predicted similarly to the validation dataset. Removing outliers and clipping helped reduce overfitting. While the model is still slightly overfitting, it performs better on unseen data compared to previous models. This model has been saved and is ready for deployment phase
	validation	92.61	
	testing	87.72	

6.b.4. Student D (Ratana Sovann)

The performance of all models trained is summarized below:

ML Algorithm	RMSE		Key Learning
Linear Regression	Training	138	• The result from this model can be used as a benchmark for other models
	Validation	138.20	
KNN Regressor	Training	-	• The experiment determines that a distanced based model such as KNN does not fit the task.
	Validation	-	
	Testing	-	
XGBoost (Best)	Training	**85.38**	• Strong performance and consistent across all sets • Training time is lower compared to KNN regressor but still moderate. • Next we will explore LightGBM which should train faster than XGBoost
	Validation	**85.91**	
	Testing	**86.22**	
LightGBM	Training	97.45	• Despite superior training speed performance fall short of XGBoost
	Validation	97.51	
	Testing	97.82	
Extratrees	Training	85.90	• Took longer time to train compared to LightGBM and XGBoost • Relatively similar performance
	Validation	87.18	
	Testing	87.60	

XGBoost performs the best followed by Extratrees whose performance is very close to XGBoost. The result is fairly consistent with my peers suggesting that the model RMSE score in the range of 85 - 95 is closest to the best achievable with the features used, giving us the confidence that this result is accurate.

c. Business Impact and Benefits

In the data exploration stage, we have seen that the median total airfare was about $345. The average RMSE of all our models were about 85 meaning on average, the model's prediction deviates by about $85 which is about 24% of the median value. The deviation might be considered significant in the airline context as it provides a reasonable estimate of day-to-day price fluctuations.

There is definitely room for improvement since we have not taken peak and off-peak dates into consideration. Additionally, impact of local holidays, features such as duration and precise trends based on intermediate airports, were not taken into consideration. However the model does reflect the impact of prices getting higher as cabin class improves, as distance increases and as flight date approaches the search date.

This model directly addresses the challenges related to price variability and identifies opportunities for targeted pricing strategies and marketing campaigns.

d. Data Privacy and Ethical Concerns

Data Privacy

The attribute *legId* would potentially impose the data privacy concerns when it is associated with or referenced to other datasets that contain sensitive information or user profiles. Added to that, the dataset in this project entailed detailed routes, including information of each leg and layover. This would reveal the user preferences. When the data was illegally used or obtained by an unauthorized party, it could possibly expose user sensitive information.

Ethical Concern

Owing to the unknown data collection method, if the data was retrieved without informed consent from users may raise ethical concerns. And users might be unaware of the usage of the data. Therefore, it is essential to obtain explicit consent from the users when collecting data. Users must be aware of and thoroughly understand whether the data gathered is anonymous, how the data will be processed, stored and used, who will be granted access to the data. Furthermore, the business needs to maintain transparency throughout the project and establish data retention policy and consequences for any violations. Another concern may cast over biases present in the data, resulting in biased and inaccurate predictions towards the imbalanced data.

Address steps taken

As *legId* represented each itinerary, the attribute was eliminated prior to model training to prevent any identification or association with other dataset. In light of addressing the bias issue, the team

had divided the data into four segments based on airports in chronological order when commencing the project. Each member processed data independently. Each dataset was partitioned into train, validation and test sets, and different algorithms were chosen. Regarding model deployment, the backend API was clearly defined and linked to the specific model that responded to the user's choice of interest on the Streamlit APP. This can ensure the consistency and robustness of model performance, while mitigating bias in the model. Last but not least, only the significant features were used to train the models and defined as requisite input parameters for predictions. For instance, details of interim legs were excluded from training, which can lessen the potential risk of breaching data privacy and ethical concern.

6. Deployment

a. Model Serving

We will be integrating FastAPI with a Streamlit Application to significantly enhance the benefits of prediction models by enabling real-time predictions with a friendly user interface. The deployment processes are outlined as follows:

1. **Model Training:** train 4 separate models for different Airports sets using past flight data and save the trained model as a joblib file including the required preprocessing steps.
2. **Model Export:** export model in a serialized format (e.g joblib) that can be loaded in the backend.
3. **Backend Development:** use FastAPI framework to create API endpoints for real-time prediction. Write a Dockerfile for the backend, containerizing the API and model so it can be deployed easily.
4. **Frontend Development:** develop the frontend using Streamlit to create a simple UI and integrate it with the backend. The frontend is also Dockerized so it can be deployed independently.
5. **Model Hosting:** use Render free tier service to host both backend and frontend with different ports.

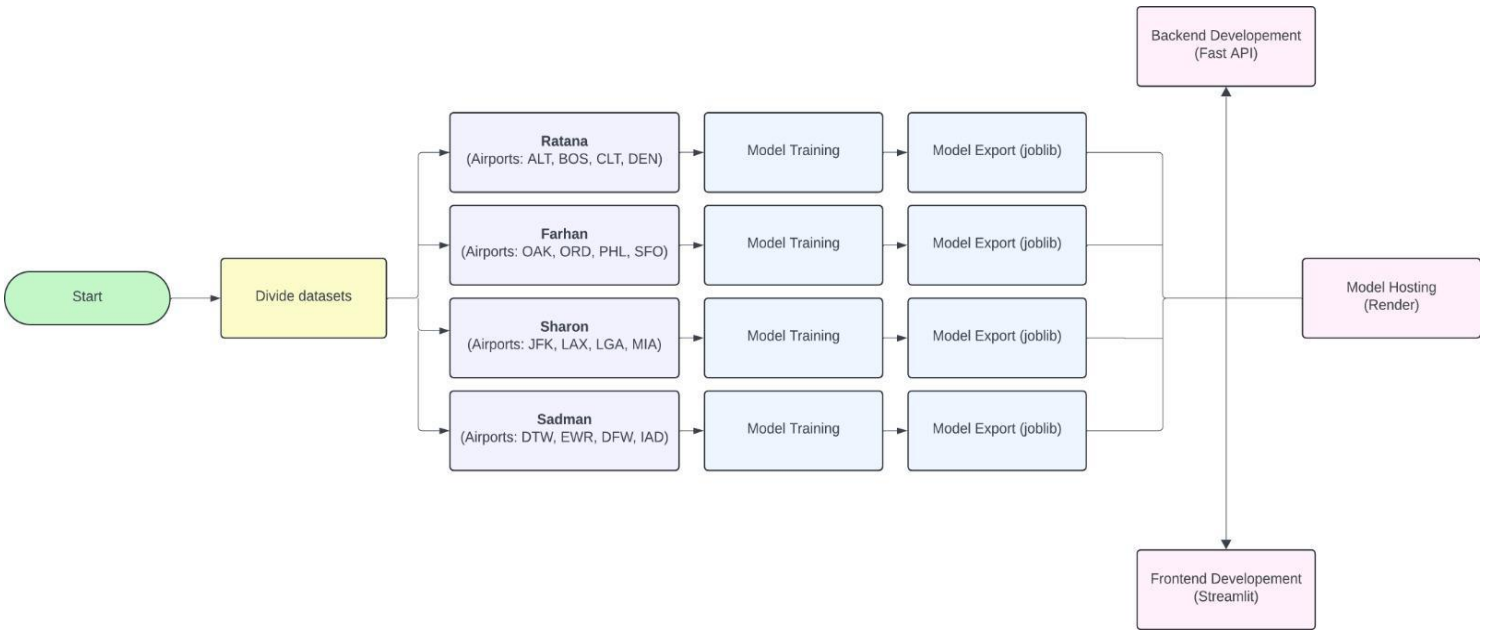


Figure 7.1: Deployment FlowChart

Challenges and Consideration for Deployment:

- **Model performance with real-world data:** The model performance on actual, unseen data might differ from those in the training set. Continuous monitoring of the model is essential in the long run to retain the accuracy of the model. At this stage two integral components of MLOps principals are missing, model monitoring and automated model retraining both of which are important for the success of model deployment.

- **Server Load from Multiple API Requests:** High traffic, especially when multiple users are making predictions simultaneously, can put stress on the server, especially on Free tier render.

b. Web App

The application aims to build a data product that will help users in the USA to better estimate their local travel airfare. The user will input: Departure_date / DepartureTime / Origin_airport / Destination_airport / Cabin Type(s) and n_stops and the model will return the predicted fare based on these parameters.

Instruction for Running the Application:

1. Docker images for both frontend and backend files need to be created and pushed to DockerHub with their specific repo. (link)

2. Models are deployed with Render which automatically link with latest Docker image pushed to Dockerhub:

   • To interact with frontend: https://at3-app-frontend-latest.onrender.com
   • To interact with backend: https://at3-app-backend-latest.onrender.com

Potential Users:

- **Travelers:** Individuals planning a trip can use the app to estimate flight fares based on selected dates, times, and other preferences.

- **Travel Agents:** Agencies can quickly predict fares for clients, especially for customized flights with specific routes, cabin types, and stops.

- **Airline Companies:** Airlines could integrate this as a customer-facing feature to help users plan their trips and optimize sales strategies based on fare predictions.

The application could be expanded commercially by integrating API that other platforms could embed, especially for agencies such as Expedia, Agoda, Booking.com to help users plan their trips. If more functionalities are incorporated such as a graph showcasing deviation of flight fare projection, automatic recommendation on the most cost-effective time to purchase a ticket or an alert system that notify user if the fare for their planned trip will likely decrease or increase, it can be a standalone app with subscription based business model for frequent travelers or agencies needing consistent fare predictions.

Limitations and Caveats:

- The current model relies on median distance and duration calculation which improved prediction results but can be further enhanced by integrating API to get exact duration based on route.

- The number of stops is not calibrated based on route. (i.e. the user can select 4 number of stops between ATL-BOS). Ideally, route should be optimized or allow users to customize their route and reflect in the number of stops that can be selected.

- Steamlit timebox logic has limited selection (5 min increments) which may not be best in terms of user experience.

7. Collaboration

a. Individual Contributions

Team Member	Contributions
Yee Ting Li	● Shared Python script and decompressed csv files for the archived datasets ● Communicate findings and solve issues with the team ● Built backend, frontend prototype and docker file for deployment ● Formatting the final report ● Final report - data preparation, experimentations, evaluation, data privacy, group dynamic
Farhan Faiyaz	● Created Github repo for collaboration ● Communicate findings such as impact of total median distance on model performance ● Build backend for model ● Formatting the final report ● Final report - executive summary, business understanding, data preparation, experimentations, evaluation, ways of working together
Md Sadman Sakib	● Build backend for model ● Final report - data understanding, data preparation, experimentations, evaluation
Ratana Sovann	● Create Github repo for the App ● Backend and frontend development integrating all the models together ● Create Docker images and Docker repo for the APP ● Handle the deployment of the app ● Final report - experimentations, evaluation, deployment, conclusion

b. Group Dynamic

Effective teamwork is crucial to the project's success. Our team had established a positive dynamic with efficient communication and supportive collaboration. Commencing with face-to-face meetings, the project goals and objectives were clearly defined, alongside the initial approach to be taken and responsibilities. Moving forward, various virtual platforms including Zoom, Google Meet and WhatsApp were utilized to foster seamless communication. Each team member was open to discuss and share the ideas and challenges encountered. In particular, WhatsApp was the primary communication channel, the minor and technical issues were addressed promptly. Furthermore, weekly online meetings were conducted to update each member's progress to ensure everyone aligned with the objectives and the milestones were met. Openness and proactive communication within the team contributed to the project's success.

c. Ways of Working Together

Methodologies and frameworks:



Fig: CRISP-DM Framework *(Hotz and Saltz)*

For this project adopted a combination of CRISP-DM and Agile methodologies to structure our workflow, promote collaboration and ensure adaptability to evolving project needs. We ensured that we clearly defined the objectives, target users and KPIs initially before moving onto the data understanding stage where we conducted EDA and assessed the quality of the data. Afterwards we cleaned the data quickly by adopting the agile methodologies to deploy the model as quickly as possible. When the initial deployment was completed, we went back and fine-tuned our models and explored more efficient algorithms. This iterative approach ensured that we can steadily enhance our model performance without stressing out at the end by focusing too much about perfecting the model in the initial stage.

Frequency and Format of Team Meetings:

We scheduled weekly meetings on Zoom to ensure all the team members are aligned with the latest findings. Any new findings and model performances were updated on WhatsApp to ensure team members are up to date with any key aspects. Each team member took initiative to lead a particular segment of the project; let it be feature engineering or deployment, enhancing the report or scheduling regular meetings. Thereby we were able to quickly get our model up and run very quickly.

Tools for Collaboration:

GitHub was used for version control, allowing us to collaborate on code, share model scripts and merge changes effectively. Steamlit was chosen for deploying our app and was deployed using Render to allow users to seamlessly try out our model. Zoom and WhatsApp were our key communication platforms.

d. Issues Faced

- Initially, the major issues were extracting multiple archived data files and the large data size for the combined set. The problems were resolved by decompressing the files iteratively and partitioning into smaller chunks. Each team member focused on 4 distinct airports.

- Large number of duplications in the dataset. We decided to drop the duplicated set on the assumption that the Expedia dataset contains repeated search of the same flight leg.

- Limitations in incorporating training features that the user cannot input such as layover time, durations that have high correlation with fares. Compromised by incorporating a csv table of precomputed medianDistance and medianDuration between all airports combinations. Ideally, the addition of Feature Store with pre-trained features ready to access via API should be used to include features that cannot be specified by the users. Some of the features that can improve the model significantly: projected layover time, optimised route

- Steamlit simplistic functions limit the capacity to decorate the frontend that makes the user experience more enjoyable.

8. Conclusion

The team has successfully created a web application that enables users to input details about their trips and return the estimated flight fare as per the stakeholders requirement. There is still room for improvement both on the technical side and user experience front. In the future, the addition of pre-trained features taking into account holiday and off-peak seasons will likely improve the prediction by a lot. Setting up infrastructure for monitoring and automatic retraining of the model are key considerations as part of the MLops process for the project scalability and commercialization.

Overall, this project has been a great learning experience showcasing the ability to create a Data product that encompasses both machine learning methodologies as well as design thinking, from backend model developments to user-friendly applications that non-technical users can utilize and interact with.

9. References

Hotz, Nick, and Jeff Saltz. “What is CRISP DM?” *Data Science Process Alliance*, https://www.datascience-pm.com/crisp-dm-2/. Accessed 7 November 2024.